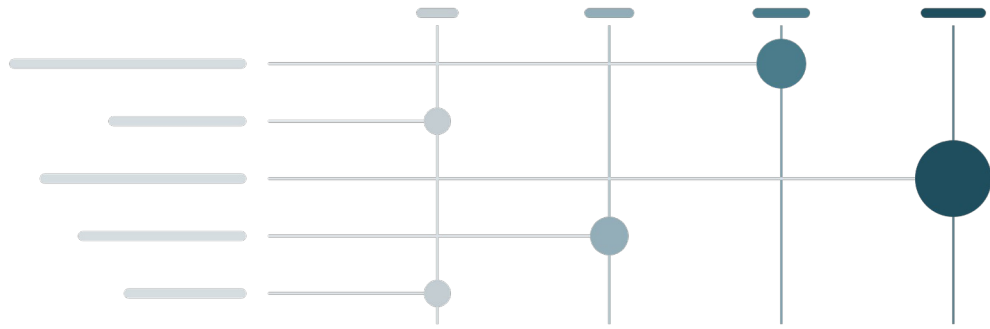




Un agent, plusieurs modèles

Modèles frontier, modèles à poids ouverts, et un modèle par rôle.
Vocabulaire, arbitrages de conception et chiffres.

Denis Lamard

Concepteur et intégrateur d'agents IA

github.com/denislamard

Version 1.1, 2026

Sommaire

1. La question qu'on me pose en premier
2. Le vocabulaire, et ce qu'il recouvre vraiment
3. Un agent n'a pas un usage, il en a 6
4. Les 3 façons de consommer un modèle
5. Pourquoi un petit modèle suffit sur la plus grande partie du travail
6. Où le petit modèle cède
7. Les 5 raisons de mélanger les modèles
8. Un rôle, un modèle, une ligne de configuration
9. Décider par la mesure
10. Vivre avec des modèles qui changent tous les 3 mois
11. Ce que je retiens
12. À propos

Annexe A. Lexique

Annexe B. Relevé des modèles et des tarifs

Annexe C. Servir un modèle sur votre infrastructure

1. La question qu'on me pose en premier

« Vous utilisez quel modèle ? »

La question arrive au premier appel, souvent avant celle du prix. Elle attend le nom d'un modèle. La question porte en réalité sur une décision d'architecture, et la réponse honnête tient en une phrase : mes agents en production utilisent 4 modèles, et le plus cher des 4 traite moins d'un cinquième du volume.

Ce document explique pourquoi. Il défend une position plutôt qu'il n'énumère des modèles. **Le choix qui compte porte sur le découpage d'un agent en rôles, et il se prend avant de choisir un nom de modèle.** Un agent construit ainsi survit aux modèles qu'il utilise, encaisse les baisses de tarif sans réécriture, et laisse la porte ouverte à un fournisseur qui n'existe pas encore.

La partie 2 pose le vocabulaire. Elle est là parce que les mots employés dans ce domaine recouvrent des réalités très différentes selon qui les prononce, et parce que plusieurs décisions d'achat se prennent sur des malentendus de définition. Un lecteur déjà au fait peut la parcourir et reprendre à la partie 3.

Les chiffres cités viennent de mes propres agents en service. Ils sont donnés comme ordre de grandeur défendable, et ils se remplacent par une mesure sur vos dossiers dès le cadrage.

2. Le vocabulaire, et ce qu'il recouvre vraiment

2.1 Ce qu'est un modèle

Un modèle de langage est un fichier de nombres, appelés **poids** ou **paramètres**, et un programme qui sait s'en servir pour prédire la suite d'un texte. Le nombre de paramètres est ce que désigne le « B » qu'on lit dans les noms de modèles : 7B pour 7 milliards, 70B pour 70 milliards.

Ce nombre a longtemps servi de mesure de qualité. Il a cessé de la prédire, pour deux raisons qui méritent d'être connues avant de comparer 2 modèles.

Le post-entraînement compte autant que la taille. Un modèle passe par un pré-entraînement, qui lui donne sa connaissance du langage, puis par une phase d'ajustement : apprentissage supervisé sur des exemples de bonnes réponses, puis apprentissage par renforcement sur des préférences. 2 produits issus du même modèle de base, ajustés différemment, se comportent différemment dans un agent. L'un suivra scrupuleusement un format de sortie, l'autre bavardera autour. La taille n'explique rien de cet écart.

La distillation a changé le paysage des petits modèles. Un grand modèle produit des réponses, un petit modèle apprend à les reproduire. Le petit hérite d'une partie du comportement du grand pour une fraction du coût d'exécution. C'est ce mécanisme qui

explique la génération actuelle de modèles compacts capables de tenir des tâches qui demandaient un grand modèle deux ans plus tôt.

La conséquence pratique est directe. La frontière entre ce qu'un petit modèle sait faire et ce qui exige un grand modèle se déplace vers le haut chaque année. Une décision d'architecture prise il y a dix-huit mois mérite d'être rejouée.

2.2 Mélange d'experts, et pourquoi la taille affichée ne dit plus le prix

Une bonne partie des modèles récents sont des **mélanges d'experts**, ou MoE. Le modèle contient plusieurs sous-réseaux spécialisés, et un routeur en active seulement quelques-uns à chaque token produit. D'où 2 nombres à distinguer :

- Les **paramètres totaux**, qui décident de la mémoire nécessaire pour charger le modèle.
- Les **paramètres actifs**, qui décident du calcul fait à chaque token, donc de la vitesse et du coût.

Un modèle annoncé à plusieurs centaines de milliards de paramètres totaux peut n'en activer qu'une trentaine, et coûter en inférence le prix d'un modèle de cette taille-là. C'est la raison pour laquelle un chiffre de paramètres ne permet plus de déduire un prix, et pour laquelle un modèle à poids ouverts très volumineux peut rester hors de portée d'une machine unique tout en restant économique chez un hébergeur qui répartit sa charge.

2.3 Tokens, et le surcoût du français

Un modèle ne lit pas des caractères, il lit des **tokens** : des fragments de mots produits par un **tokeniseur**. La facturation se fait au million de tokens, séparément en entrée et en sortie.

Le point qui concerne directement un lecteur francophone tient au fait que les tokeniseurs de la plupart des modèles sont ajustés sur des corpus majoritairement anglophones. Un même contenu consomme davantage de tokens en français qu'en anglais, avec un écart courant de l'ordre de 15 à 30 % selon le vocabulaire employé. Ce surcoût s'applique à chaque appel, sur l'entrée comme sur la sortie, et il se retrouve intégralement sur la facture mensuelle. Il mérite d'être intégré à toute projection faite à partir de tarifs annoncés.

2.4 Ce que « frontier » désigne

Frontier qualifie les modèles qui, à un instant donné, définissent le **plafond de capacité accessible**. Le mot décrit une position dans le temps. Un modèle frontier de 2024 est un modèle courant de 2026, et le même nom de produit change de catégorie sans que rien n'ait changé en lui.

Ce qui les caractérise durablement tient moins à leur qualité qu'à leur mode de distribution :

- **Poids fermés.** Le fichier de paramètres reste chez le fournisseur. L'accès se fait par une API, sur son infrastructure.
- **Tarification asymétrique.** Le prix du million de tokens en sortie est couramment trois à cinq fois celui de l'entrée. Un rôle qui lit beaucoup et écrit peu, comme un juge ou un classifieur, a donc un profil de coût très différent d'un rôle de rédaction, sur le même modèle.
- **Dépréciation subie.** Le fournisseur annonce puis retire des versions. Une version datée doit être épinglée dans la configuration, sous peine de voir le comportement du système changer un matin sans intervention de votre part.
- **Vos données partent chez un tiers,** avec les conséquences contractuelles traitées au point 4.

Une sous-famille mérite d'être isolée, celle des **modèles de raisonnement**. Ils produisent, avant de répondre, une séquence de tokens de réflexion qui n'apparaît pas dans la réponse et qui est facturée, souvent au tarif de sortie. Le budget de réflexion est réglable. Sur un rôle d'orchestration, il peut multiplier le coût d'un appel par un facteur significatif, et un tarif affiché ne suffit plus à comparer deux modèles sans connaître ce budget.

2.5 Poids ouverts, source ouverte, et ce que dit la licence

C'est la distinction la plus souvent escamotée, et celle qui a le plus de conséquences juridiques.

La quasi-totalité des modèles appelés « open source » sont des modèles à poids ouverts. Le fichier de paramètres est publié et téléchargeable. Les données d'entraînement, la recette et le code d'entraînement restent chez leur auteur. On peut exécuter le modèle et l'ajuster, sans pouvoir le reproduire ni auditer ce sur quoi il a été entraîné. Une poignée de projets publient l'ensemble, données comprises, et méritent alors le terme de source ouverte.

Cette précision compte au-delà du vocabulaire, parce qu'elle décide de ce qui est vérifiable. Une question de conformité sur la présence de données personnelles ou d'œuvres protégées dans un corpus d'entraînement se pose de la même façon à un modèle à poids ouverts et à un modèle fermé.

Ce que la licence accorde se lit en trois droits distincts, qu'une même licence n'accorde pas toujours ensemble.

Droit	Ce qu'il permet
Exécution	Faire tourner le modèle sur votre infrastructure
Modification	L'ajuster sur vos données, le distiller, le fusionner
Exploitation commerciale	Le mettre au service d'une activité facturée

3 régimes se rencontrent en pratique. Les licences logicielles classiques, Apache 2.0 ou MIT, accordent les trois sans condition notable. Les licences dites communautaires accordent les trois avec des réserves : restrictions d'usage énumérées, obligation de mention, parfois un seuil d'utilisateurs au-delà duquel un accord séparé devient nécessaire. Les licences de recherche accordent l'exécution et la modification, et réservent l'exploitation commerciale.

La vérification prend dix minutes et se fait une fois par modèle retenu. Elle appartient au dossier de conformité au même titre que le contrat de sous-traitance.

2.6 Ce qu'un agent exige, et que les classements publics mesurent mal

Un agent demande au modèle trois capacités distinctes, et la troisième décide de la faisabilité du projet.

Produire une sortie structurée. Le modèle rend un objet conforme à un schéma : champs attendus, types respectés. Deux mécanismes existent. Le mode JSON demande au modèle de bien se tenir. Le **décodage contraint par grammaire** rend mécaniquement impossible la production d'un token qui violerait le schéma. Le second est une garantie, le premier une probabilité élevée. Sur un rôle d'extraction à fort volume, la différence se voit dans les journaux.

Appeler le bon outil. Le modèle reçoit une liste d'outils décrits, et choisit. Un modèle peut produire un JSON impeccable et se tromper systématiquement d'outil, ou appeler un outil dont il a déjà le résultat dans son contexte.

Enchaîner sur plusieurs tours. Le modèle lit le résultat d'un outil, en déduit l'appel suivant, et sait s'arrêter quand il a de quoi répondre. C'est la capacité la plus coûteuse à obtenir, la plus discriminante entre modèles, et celle qui détermine si le rôle d'orchestration peut descendre en gamme.

Les classements publics rendent mal compte de la troisième, pour trois raisons. Les jeux d'évaluation les plus cités finissent par se retrouver, en partie, dans les corpus d'entraînement des modèles qui les passent. Une tâche de benchmark ressemble peu à un

rôle réel branché sur vos données et vos outils. Et les classements par vote humain mesurent une préférence de style autant qu'une capacité.

Ce qui les remplace utilement est modeste et local : une trentaine de cas issus de votre propre processus, avec le résultat attendu, rejoués à chaque changement. C'est l'objet de la partie 9, et c'est le seul chiffre qui décide chez moi d'une bascule de modèle.

3. Un agent n'a pas un usage, il en a plusieurs

On parle du « modèle de l'agent » comme s'il y en avait un. Un agent qui traite une demande enchaîne des tâches dont les exigences n'ont rien de commun. En voici la liste, telle qu'elle se retrouve sur la plupart des intégrations que je livre.

Rôle	Ce qu'il fait	Ce qu'il exige
Orchestration	Choisit les outils, lit leurs résultats, décide de la suite	Enchaînement sur plusieurs tours, fiabilité de l'appel d'outils, longue fenêtre
Classification	Range une entrée dans une catégorie connue	Sortie contrainte, faible latence
Extraction	Sort des champs typés d'un texte libre	Respect strict d'un schéma, faible température
Rédaction client	Produit le texte qui partira au destinataire	Qualité de langue, tenue du registre
Juge	Évalue une sortie critère par critère	Lecture attentive, sortie contrainte, coût faible car appelé à chaque exécution
Compaction	Résume les tours anciens d'une session	Fidélité aux faits, coût faible car appelé souvent
Vision	Lit une photo, un scan, un document	Capacité multimodale, appelée selon le besoin

Confier le même modèle à ces sept rôles revient à payer le prix de l'exigence la plus forte sur la totalité du volume. C'est le choix par défaut de la plupart des prototypes, et il tient tant que le volume reste faible.

Le chiffre qui décide. Sur mon agent de relance de devis, la ventilation du coût par rôle donne ceci : les trois étapes d'orchestration pèsent 76 % de la facture, les rôles spécialisés qui font le travail visible en pèsent 17 %, et le contrôle qualité 7 %.

La lecture est contre-intuitive. Le rôle qui coûte est celui qui décide, parce qu'il voit tout le dossier, relit chaque résultat d'outil, et recommence à chaque tour. C'est là que se trouve le levier, et c'est aussi le rôle qui supporte le moins bien une descente en gamme. Toute la suite tourne autour de cette tension.

4. Les 3 façons de consommer un modèle

La distinction utile porte moins sur les poids que sur le mode de consommation. Trois modes, avec des prix, des contraintes et des engagements différents.

4.1 L'API d'un fournisseur frontier

Ce qu'on achète : le niveau de capacité du moment, une disponibilité opérée par quelqu'un d'autre, et une mise à jour continue.

Ce qui vient avec : une dépendance à un calendrier de dépréciation qui n'est pas le vôtre, une variabilité de latence aux heures de charge, des limites de débit à négocier, et un transfert de données vers un tiers à traiter au titre du RGPD.

C'est le bon choix pour le rôle d'orchestration de la plupart des agents, et pour tout ce qui demande le meilleur enchaînement disponible.

4.2 L'API d'un hébergeur de modèles à poids ouverts

Le même confort d'usage, sur des modèles publics, à un tarif qui se situe couramment très en dessous des offres frontier. Le marché compte plusieurs acteurs, dont certains se distinguent par la latence et d'autres par le prix. Deux points de vigilance.

Un endpoint compatible n'est pas le modèle d'origine. Deux hébergeurs servant le même modèle peuvent le servir quantifié différemment, avec des paramètres par défaut différents et des fenêtres de contexte différentes. Un jeu de cas rejoué après un changement d'hébergeur reste nécessaire, même quand le nom du modèle est identique.

La grille tarifaire est un fichier de configuration, jamais une constante du code. Les prix de cette catégorie bougent plus vite que vos versions logicielles.

4.3 Sur votre infrastructure

Le modèle tourne sur un GPU que vous louez ou que vous possédez. Les détails d'exécution, quantisation et service, sont en annexe C.

Le motif d'y aller est presque toujours la localisation des données, et le prix vient en second. Un secteur régulé, une clause client qui interdit la sortie de données, un dossier de conformité qui bloque : ces situations se débloquent par un rôle exécuté en interne, souvent un seul, pendant que les autres continuent d'appeler des API.

Le calcul économique, lui, se pose ainsi. Une API se paie au token consommé. Un GPU se paie à l'heure, qu'il travaille ou non. Le seuil de bascule s'obtient en divisant le coût mensuel du GPU par le prix au million de tokens de l'API équivalente, ce qui donne le volume mensuel au-delà duquel l'infrastructure dédiée devient moins chère.

Trois corrections à apporter à ce calcul avant de le présenter.

- Le débit réel du GPU sur votre modèle quantisé, mesuré en tokens par seconde sous charge concurrente, borne le volume qu'il peut absorber. Un seuil de bascule qui dépasse cette capacité désigne une machine, ou un cluster.
- Le temps d'exploitation compte : mise à jour, supervision, reprise après incident, et la personne qui s'en charge.
- La disponibilité d'une machine unique est inférieure à celle d'une API. Un rôle critique exécuté en interne demande une solution de repli, ce qui ramène en général à une configuration mixte.

En pratique, la bascule complète vers l'auto-hébergement se justifie sur de gros volumes stables. La bascule partielle, un rôle ou deux, se justifie beaucoup plus tôt, et pour des raisons de conformité.

4.4 Le tableau de synthèse

	API frontier	API poids ouverts	Votre infrastructure
Coût	Le plus élevé	Faible au token	Fixe, indépendant du volume
Mise en service	Immédiate	Immédiate	Jours à semaines
Où vont vos données	Chez le fournisseur	Chez l'hébergeur	Chez vous
Sous-traitance RGPD	À contractualiser	À contractualiser	Sans objet
Maîtrise de la version	Le fournisseur décide	Vous choisissez le modèle	Totale, gelée si vous voulez
Capacité maximale	La plus haute du marché	Une génération derrière	Bornée par votre matériel
Exploitation	Aucune	Aucune	À votre charge

5. Pourquoi un petit modèle suffit sur la plus grande partie du travail

Voici le raisonnement central du document, et il tient en une observation.

Quand une sortie est contrainte par un contrat vérifié avant tout effet, la fiabilité vient du contrat. Le modèle devient alors largement interchangeable sur ce rôle, et l'écart de prix se prend en entier.

Une extraction de 6 champs typés, validée contre un schéma, échoue de façon visible et re-jouable si elle échoue. Une rédaction dans un gabarit fermé se vérifie de la même

manière. Sur ces rôles, payer un modèle de raisonnement revient à acheter une capacité que la tâche n'appelle pas.

Six tâches où le rôle rapide tient, avec la raison pour chacune.

Classer dans des catégories fermées. L'espace de sortie compte quelques valeurs, et un décodage contraint interdit toute autre réponse. La difficulté porte sur la lecture. Un cas ambigu se traite par une catégorie « à escalader » plutôt que par un modèle plus puissant.

Extraire des champs contre un schéma. L'information est présente dans le texte, la tâche consiste à la transcrire dans une forme. Le schéma attrape les défauts de forme, le juge attrape les valeurs inventées, ce qui laisse au modèle une marge de manœuvre étroite. Température à zéro, sortie courte, coût minime.

Tenir le rôle de juge. Le juge lit beaucoup et écrit peu, ce qui le place du bon côté de l'asymétrie tarifaire. Il évalue critère par critère, avec des critères écrits au cadrage, et rend un verdict structuré. Un point de conception vaut d'être respecté ici : le juge tourne sur un modèle différent de celui qu'il évalue, sans quoi il valide ses propres biais.

Compacter un historique de session. Résumer des tours anciens en gardant les décisions, les montants, les dates et les engagements pris. La fidélité prime, la finesse d'écriture ne sert à rien. Ce rôle est appelé souvent et sur de longues entrées, ce qui en fait un candidat évident.

Rédiger dans un gabarit contraint. Une relance de devis suit une structure connue, un registre convenu et une longueur bornée. Le contrat de sortie interdit les montants et les engagements de délai. Ce qui reste au modèle est un travail d'habillage, que la génération actuelle de modèles rapides tient très bien en français.

Reformuler et traduire. Tâche à entrée et sortie de même longueur, sans décision. Le rapport qualité-prix y est le plus favorable de la liste.

5.1 Ce que ça donne en euros

Le même agent de relance, configuré de cinq façons, à volume constant et à qualité contrôlée par les mêmes critères de juge.

Configuration	Par relance	Par mois (800)	Indice
Un modèle haut de gamme sur tous les rôles	0,0979 €	78 €	167
Un modèle intermédiaire sur tous les rôles	0,0588 €	47 €	100
Le même, en génération courante	0,0392 €	31 €	67
Un modèle adapté à chaque rôle	0,0317 €	25 €	54

Configuration	Par relance	Par mois (800)	Indice
Idem, avec la consigne mise en cache	0,0277 €	22 €	47

L'écart entre la première et la dernière ligne est de 3,5. Ce sont les mêmes cas, la même qualité de sortie mesurée sur les mêmes critères, et une seule différence : la répartition des rôles sur les modèles.

Deux enseignements se dégagent de ce tableau, et l'ordre compte.

Le premier levier est la répartition. Passer d'un modèle unique à un modèle par rôle divise la facture par près de deux. Ce levier se prend une fois, à la conception, et il continue de produire ensuite.

La mise en cache de la consigne vient loin derrière. La partie fixe de l'instruction, le rôle, les règles, la description des outils, est réexpédiée à chaque tour. La faire mettre en cache ramène son prix à une fraction. Le gain sur cet agent est de 13 % : réel, et bien plus modeste que celui du premier levier. C'est un bon exemple de ce que la mesure apporte face à l'intuition, puisque la mise en cache est l'optimisation qu'on cite en premier dans les conversations techniques.

5.2 La longueur de conversation, cas particulier

Sur un agent conversationnel, un facteur s'invite : un modèle n'a pas de mémoire propre. À chaque question, **toute la conversation précédente lui est réexpédiée**. Le premier échange transporte une page, le vingtième en transporte quatorze.

Le coût suit. Sur une conversation de vingt échanges, le premier coûte 0,52 centime et le vingtième 3,98, pour une question de longueur identique. La compaction par un petit modèle ramène l'ensemble de la conversation de 45 à 30 centimes, coût du résumé compris, en gardant les décisions, les montants et les engagements pris.

C'est le rôle où le petit modèle rapporte le plus, et c'est celui qu'on oublie le plus souvent d'attribuer.

6. Où le petit modèle cède

Cinq situations. Elles se reconnaissent au symptôme plus qu'à un score, et le symptôme est rarement une erreur franche.

L'enchaînement d'outils sur plusieurs tours. Le rôle d'orchestration est le dernier à céder du terrain. Le symptôme est la boucle : l'agent appelle un outil dont il a déjà le résultat, repose une question à laquelle il a répondu, ou consomme ses vingt-cinq tours sans converger. Il coûte alors plus cher que le modèle qu'on voulait éviter.

L'arbitrage sur un cas ambigu. Une demande qui contient plusieurs intentions, une pièce jointe qui change le sens du message, un client dont l'historique contredit ce qu'il écrit. Le symptôme est le traitement de la première intention seule, avec une réponse par ailleurs correcte, ce qui la rend difficile à repérer autrement que par échantillonnage humain.

La sortie qui part telle quelle au client. Le registre, la nuance et l'à-propos échappent à un contrat de forme. Un juge attrape les écarts flagrants, et il attrape mal un texte poli et légèrement à côté. Sur ce rôle, l'économie se paie en image.

La consigne longue avec des exceptions. Un prompt système qui énumère quinze règles dont trois exceptions conditionnelles. Le symptôme est le respect des premières règles et l'oubli progressif des dernières, avec une dérive qui s'accroît à mesure que la conversation s'allonge. La parade tient rarement au modèle : elle consiste à sortir les règles du prompt et à les écrire dans le code, où elles deviennent des contraintes.

Le champ inventé. Ce symptôme mérite d'être isolé parce qu'il se voit le moins. La sortie est bien typée, le schéma passe, et la valeur est fautive : un numéro de dossier plausible, un montant cohérent avec le contexte et absent du document. Aucune validation de forme ne l'attrape. C'est précisément le travail du juge sémantique, et c'est l'argument le plus fort en faveur de son existence sur les rôles d'extraction confiés à un modèle rapide.

7. Les 5 raisons de mélanger les modèles

Le coût est la raison la plus visible. Le coût est loin d'être la seule raison.

1. Le coût. Traité au point 5. Un facteur de 3,5 sur la facture des modèles, pris à la conception, à qualité mesurée constante.

2. La latence. Sur un agent conversationnel, la latence du premier token décide du ressenti bien plus que la qualité rédactionnelle. Confier la classification et l'extraction à un modèle rapide raccourcit le chemin critique avant que l'utilisateur n'ait perçu quoi que ce soit du modèle principal. Sur un agent vocal, ce point cesse d'être un confort et devient une condition de faisabilité.

3. Le réglage par rôle. Deux rôles peuvent partager le même modèle avec des paramètres opposés : une extraction à température zéro, une rédaction à 0,7, deux plafonds de tokens différents. Le réglage est un choix de rôle avant d'être un choix de modèle, et une architecture qui ne le permet pas oblige à un compromis médiocre pour tout le monde.

4. La résilience. Un incident chez un fournisseur, une limite de débit atteinte en fin de mois, une version dépréciée. Un agent réparti sur deux fournisseurs continue de fonctionner en mode dégradé pendant que le premier revient. Cette propriété s'obtient gratuitement quand le modèle d'un rôle est une ligne de configuration, et elle se paie cher quand il faut la rajouter après coup.

5. La localisation des données. C'est la raison qui débloque le plus de projets en secteur régulé. Le rôle qui voit des données nominatives tourne sur votre infrastructure, sur un modèle à poids ouverts, pendant que les rôles qui travaillent sur des données anonymisées appellent une API. La conformité devient une propriété d'architecture, constatable dans un fichier de configuration, plutôt qu'une intention à défendre en comité.

8. Un rôle, un modèle, une ligne de configuration

Tout ce qui précède suppose que changer le modèle d'un rôle coûte peu. Sans cette condition, la démonstration reste théorique : personne ne teste cinq configurations si chacune demande une journée de développement.

Ces contraintes reviennent à l'identique sur chaque intégration. Je les ai factorisées dans **LOOM.IA**, un socle d'orchestration d'agents en Python que je maintiens en open source, disponible sur github.com/denislamard/loom. Voici les décisions de conception qui rendent le mélange praticable, indépendamment de cet outil.

Une abstraction de fournisseur, et un fournisseur qui tient dans un fichier. Une classe de base porte ce qui est commun à tous : reprise avec temporisation croissante, contrôle de la fenêtre de contexte avant l'appel, comptabilisation de la consommation, émission de la métrique. Une implémentation par format d'API. Comme la plupart des fournisseurs exposent l'un des deux formats dominants, brancher un hébergeur de modèles ouverts, un serveur local ou un nouvel acteur revient souvent à changer une URL dans la configuration.

Le modèle d'un rôle est déclaré dans un fichier validé. Un rôle se décrit par son modèle, sa consigne, sa description et ses surcharges de paramètres. Le fichier refuse les clés inconnues, ce qui fait échouer une faute de frappe au démarrage. Changer le modèle d'un rôle est une modification d'une ligne, sans recompilation ni redéploiement du code.

```
{
  "name":    "extract_fields",
  "llm":    "HAIKU",      # le modèle du rôle, ici seulement
  "description": "Extrait les six champs d'un devis.",
  "system":  "Extrais les six champs. Renvoie uniquement le JSON.",
  "llm_config": { "temperature": 0, "max_tokens": 400 },
  "terminal": true
}
```

L'historique de session est stocké dans un format neutre. Avant écriture, les images deviennent un court texte, le raisonnement des tours terminés est retiré, et les tours d'outils sont aplatis en trace textuelle. L'historique redevient une simple alternance entre l'utilisateur et l'assistant. La conséquence conditionne tout le reste : **changer le modèle du rôle principal laisse les sessions en cours fonctionnelles**, là où un historique stocké au format natif d'un fournisseur serait rejeté par un autre.

Le coût est calculé par appel, au tarif du modèle qui a fait cet appel. Quand quatre modèles cohabitent dans une même exécution, un total ne se ventile pas après coup. La mesure par appel est la seule façon d'attribuer une dépense à un rôle, donc de décider lequel descendre en gamme. Les tarifs vivent dans la configuration, puisqu'une grille change plus vite qu'une version logicielle.

Un rôle spécialisé est un outil pour l'orchestrateur. Le rôle principal voit les autres rôles sous la même forme qu'une fonction Python ou qu'un serveur distant : un nom, une description, des arguments. C'est la description qui dicte quand l'appeler, ce qui place le routage dans la configuration. Ajouter un spécialiste à un agent demande alors quelques lignes de déclaration.

La configuration de référence du dépôt public met en œuvre exactement ceci : quatre modèles, sept rôles dont six exposés comme outils du rôle principal. Le code est disponible, ce qui rend l'ensemble de ce paragraphe vérifiable avant tout engagement.

9. Décider par la mesure

Tout ce qui précède désigne des candidats. La bascule, elle, se décide sur des chiffres obtenus sur vos cas.

Le jeu de cas. Une trentaine de cas issus de votre processus, avec pour chacun l'entrée réelle et le résultat attendu. Ils se constituent naturellement pendant les premières semaines d'exploitation, et les désaccords entre l'agent et l'humain en sont la meilleure source. Ce jeu est figé : il change quand le processus change, et il reste stable quand un résultat déplaît.

Les critères du juge. Les mêmes critères servent à valider une sortie en production et à comparer deux modèles. Chacun porte son seuil. C'est ce qui permet d'affirmer qu'une bascule s'est faite à qualité contrôlée plutôt qu'à qualité supposée.

Trois indicateurs, relevés à chaque exécution du jeu : le taux d'accord avec le résultat attendu, le coût par cas, et la latence au 90e centile. La moyenne de latence cache exactement ce qui gêne les utilisateurs.

La répétition, pour repérer l'instabilité. Le même cas rejoué trois fois. Un cas qui passe deux fois sur trois porte une information différente d'un échec franc : il désigne un rôle non déterministe, qui se corrige d'abord par la température ou par la consigne. Un modèle rapide instable sur un rôle donné se disqualifie plus sûrement qu'un modèle rapide qui échoue de façon constante.

La règle de bascule. Un rôle à la fois, en commençant par le moins critique, et retour en arrière dès qu'un indicateur bouge. L'ordre que je suis : compaction, puis juge, puis extraction, puis classification, puis rédaction, et l'orchestration en dernier si elle bouge

un jour. Descendre deux rôles en même temps rend l'écart inattribuable, ce qui oblige à tout refaire.

10. Vivre avec des modèles qui changent tous les trois mois

Le rythme du marché est une contrainte d'exploitation, et il se prépare.

Épingler les versions. Un identifiant de modèle daté dans la configuration, plutôt qu'un alias qui suit la dernière version. Un alias fait changer le comportement de votre système un matin, sans intervention, et le diagnostic prend une journée.

Suivre les dépréciations. Les fournisseurs annoncent le retrait de leurs versions avec un préavis. Cette date appartient au calendrier d'exploitation au même titre qu'un renouvellement de certificat.

Rejouer le jeu de cas avant toute bascule. C'est la seule protection contre la régression silencieuse, celle qui se voit trois semaines plus tard dans un indicateur métier.

Encaisser les baisses de tarif. Chaque génération rend accessible, à prix inférieur, le niveau de qualité de la précédente. Le tableau du point 5 montre le même travail passer de 47 à 31 euros par mois d'une génération à l'autre. Une architecture où le modèle est une ligne de configuration prend cette baisse en changeant une ligne et en jouant trente cas. Une architecture où le modèle est inscrit dans le code laisse cette baisse sur la table, et l'écart se creuse à chaque génération.

C'est, à mes yeux, l'argument décisif en faveur du découpage en rôles. Le gain immédiat est mesurable et il vaut la peine. Le gain qui compte sur trois ans est la capacité à suivre un marché qui bouge tous les trimestres.

11. Ce que je retiens

Le choix du modèle est une conséquence. Ce qui se décide avant lui, et qui engage davantage, est le découpage de l'agent en rôles et la place du modèle dans l'architecture.

Un agent qui déclare son modèle dans sa configuration, qui stocke son historique dans un format neutre et qui mesure son coût par appel peut changer de modèle en une ligne et vérifier la bascule en trente cas. Il prend les baisses de tarif, il absorbe les dépréciations, il isole un rôle sensible sur votre infrastructure quand la conformité le demande, et il continue de fonctionner quand un fournisseur a un incident.

La question « vous utilisez quel modèle ? » a donc une bonne réponse, et ce n'est pas un nom. C'est : celui que le rôle appelle, et il est écrit dans un fichier que votre équipe peut lire.

12. À propos

Je conçois et j'intègre des agents IA en production. Les principes décrits ici sont mis en œuvre dans **LOOM.IA**, un socle d'orchestration d'agents en Python que je maintiens en open source : orchestration multi-modèles, un modèle par rôle déclaré en configuration, outils exécutés en parallèle, validation des sorties, comptabilité et plafonnement du coût, mémoire de session et observabilité.

Code disponible : github.com/denislamard/loom

Détail technique et cas d'usage : denislamard.github.io

Annexe A. Lexique

Appel d'outils. Mécanisme par lequel un modèle demande l'exécution d'une fonction en produisant son nom et ses arguments.

Décodage contraint. Technique qui interdit mécaniquement au modèle de produire un token qui violerait un schéma attendu. Se distingue du mode JSON, qui demande sans garantir.

Distillation. Entraînement d'un petit modèle à reproduire les sorties d'un grand.

Fenêtre de contexte. Nombre maximal de tokens qu'un modèle peut lire dans un même appel, historique et consigne compris.

Frontier. Qualifie les modèles qui définissent, à un instant donné, le plafond de capacité accessible. Position relative dans le temps.

Mélange d'experts (MoE). Architecture où seuls quelques sous-réseaux sont activés par token. Distingue paramètres totaux et paramètres actifs.

Mise en cache de la consigne. Facturation réduite de la partie fixe d'un prompt réexpédiée à chaque tour.

Modèle de raisonnement. Modèle qui produit des tokens de réflexion facturés avant de répondre.

Paramètres actifs. Paramètres effectivement mobilisés pour produire un token. Déterminent le coût et la vitesse.

Poids. Le fichier de nombres qui constitue le modèle.

Poids ouverts. Modèle dont les paramètres sont publiés et téléchargeables, sans que les données ni la recette d'entraînement le soient.

Post-entraînement. Phases d'ajustement qui suivent le pré-entraînement et fixent le comportement du modèle.

Quantisation. Réduction de la précision numérique des poids pour diminuer l'empreinte mémoire et augmenter le débit.

Token. Fragment de texte produit par le tokeniseur. Unité de facturation.

Tokeniseur. Programme qui découpe un texte en tokens. Son découpage varie selon la langue.

Annexe B. Relevé des modèles et des tarifs

Les tarifs et les modèles disponibles changent plusieurs fois par an. Cette annexe rassemble les seuls chiffres du guide qui se périment, pour que le corps du document reste exact sans être réécrit. Remplissez-la au démarrage de l'étude, datez-la, et refaites le relevé avant chaque décision d'architecture.

Rôle	Modèle retenu	Mode de consommation	Prix entrée / sortie	Fenêtre
Orchestration				
Rédaction client				
Extraction				
Classification				
Juge				
Compaction				
Vision				

Annexe C. Servir un modèle sur votre infrastructure

Cette annexe s'adresse à l'équipe technique. Le corps du document se lit sans elle.

Quantisation

Réduire la précision numérique des poids diminue l'empreinte mémoire et augmente le débit, au prix d'une dégradation de qualité qui reste modérée jusqu'à un certain niveau et devient sensible au-delà.

Les formats se distinguent par leur cible d'exécution. **GGUF**, avec ses niveaux notés Q8, Q6, Q5, Q4 et en dessous, s'adresse aux exécutions sur processeur ou sur matériel mixte et reste le format le plus répandu pour un poste de travail. **AWQ** et **GPTQ** visent l'exécution sur GPU avec un serveur d'inférence. **FP8** demande un matériel récent et offre le meilleur compromis quand il est disponible.

Le repère utile, à vérifier sur votre modèle : la dégradation reste faible jusqu'à quatre bits par poids environ, et devient perceptible en dessous, d'abord sur les tâches de raisonnement et sur la tenue des formats de sortie. Sur un rôle contraint par un schéma, elle se mesure sur votre jeu de cas plutôt qu'elle ne se déduit d'un tableau.

Dimensionner la mémoire

Trois postes composent le besoin en mémoire vidéo.

- **Les poids.** Environ 0,6 gigaoctet par milliard de paramètres à quatre bits, à doubler pour huit bits. Sur un mélange d'experts, ce calcul porte sur les paramètres totaux, puisque tout le modèle doit être chargé.
- **Le cache d'attention.** Il croît avec la longueur de contexte et avec le nombre de requêtes traitées en parallèle. C'est le poste qui surprend, et celui qui décide de la concurrence tenable.
- **Une marge d'exécution,** de l'ordre de 10 à 15 %.

Un dimensionnement se valide sous charge, avec la fenêtre de contexte réelle de votre rôle et le nombre de requêtes simultanées visé.

Servir

vLLM apporte le lot continu et une gestion paginée du cache d'attention, ce qui le place devant les autres solutions dès qu'il y a de la concurrence. C'est le choix pour un service exposé à plusieurs appelants.

Ollama et les runtimes équivalents installent et servent un modèle en quelques minutes, avec une API compatible. C'est le choix pour le prototypage et pour un usage à un seul appelant.

Les grandeurs à mesurer avant de décider : latence du premier token, débit en tokens par seconde sous la concurrence visée, et nombre de requêtes simultanées tenues avant que la latence ne se dégrade. Ces trois chiffres, relevés sur votre matériel et votre modèle quantisé, valent tous les tableaux comparatifs publiés.